

RAG 技术详解与实战应用

第5讲：打造专属Reader组件：

轻松解析HTML、PDF等复杂文档格式



目录

-  1. 上节回顾
-  2. Reader简介
-  3. 自定义Reader
-  4. 其他特殊情况下的Reader
-  5. 实战：复杂PDF解析



Git管理

- **Git简介：** Git是一个分布式版本控制系统，用于跟踪文件更改，适用于软件开发。
- **使用原因：** 代码历史可追溯、多人协作、分支管理、托管方便。
- **仓库创建：** 在GitHub上创建项目并初始化Git仓库。
- **重要文件：** README.md、.gitignore、requirements.txt、Dockerfile、setup.py。
- **分支管理：** 主分支、开发分支、功能分支、修复分支。
- **标签管理：** 语义化版本、里程碑标签、时间戳标签、环境标签。

代码质量保证

- **风格检查：** 使用flake8和black工具进行代码检查和格式化。
- **单元测试：** 使用pytest进行测试，确保代码正确性和可维护性。
- **测试覆盖率：** 使用pytest-cov生成覆盖率报告。
- **持续集成：** 使用GitHub Actions自动化测试。
- **文档系统：** 使用MkDocs搭建文档系统，托管到Read the Docs。

工程发布

- **制品：** 在Python项目开发过程中，通常需要将项目打包成可发布、可安装、可复现的格式，称为制品。
- **whl包：** 使用setuptools进行打包，生成.whl包，便于发布与分发。
- **Docker镜像：** 使用Docker制作镜像制品，保证环境一致性，便于部署和扩展。



目录



1. 上节回顾



2. Reader简介



3. 自定义Reader



4. 特殊情况下的Reader



5. 实战：复杂PDF解析

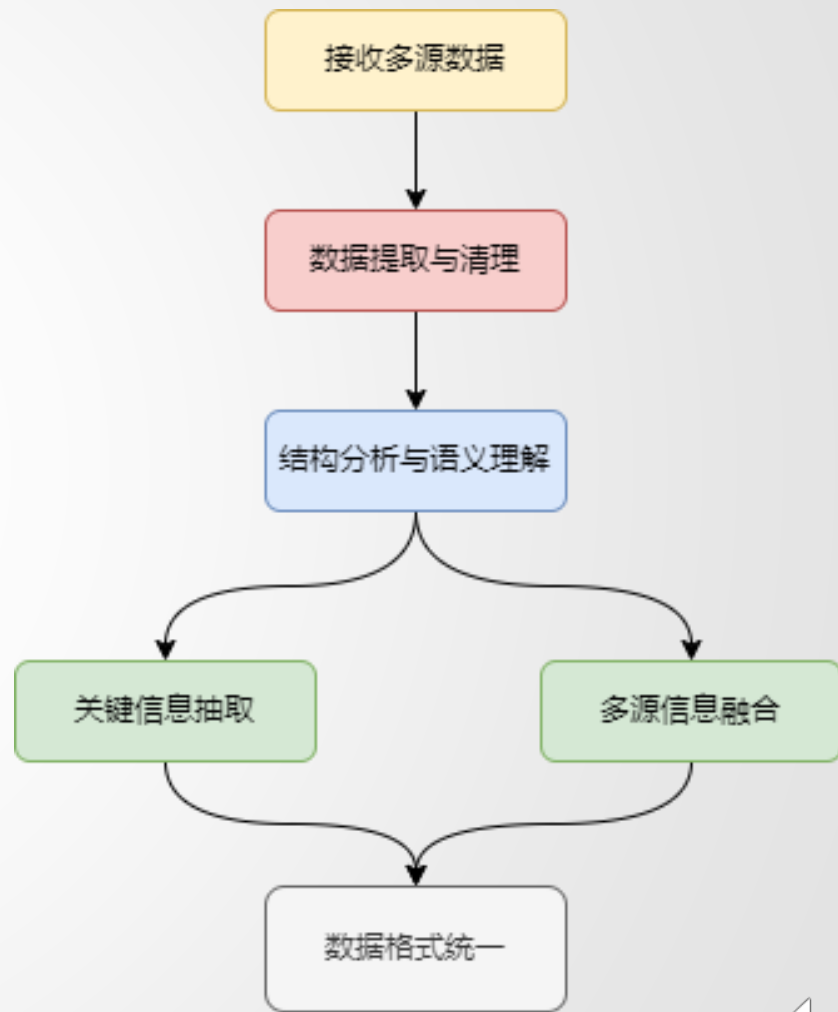


Reader 基本功能

什么是Reader?

我们都知道 Retriever 是需要从知识库中检索和问题相关的信息，而知识库中的内容会是各种各样的，有结构化存储的，也有非结构化存储的。为了让 Retriever 中的检索模块和知识库中知识的**存储格式解耦**，就需要 Reader 模块来大显身手了。

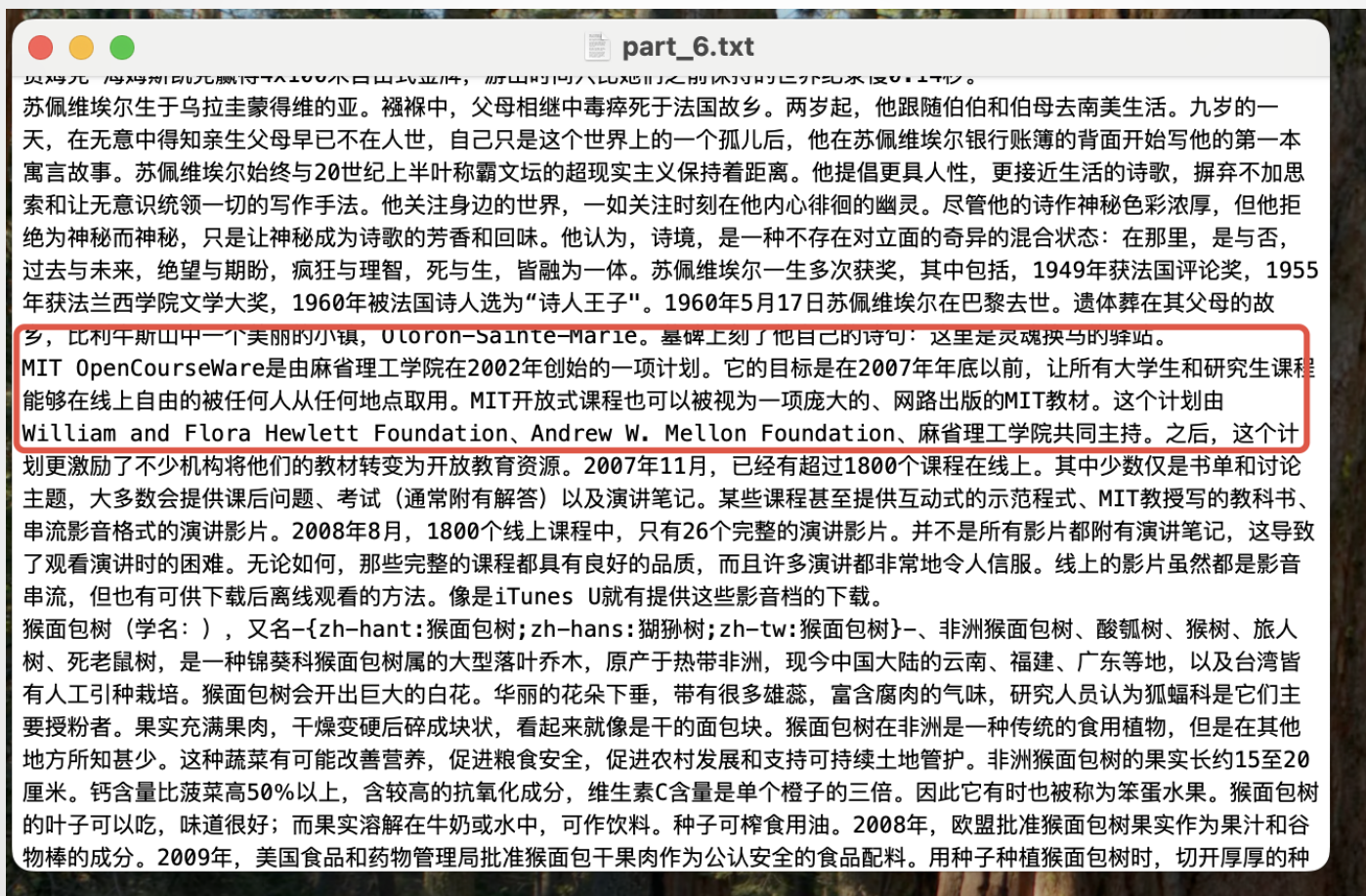
Reader 的功能就是负责把以各种形式存储的知识读取出来，然后以**统一的格式**给检索模块使用。



Reader 使用方法



接下来我们看看 Reader 的使用方法。首先，我们以 [cmrc2018](#) 知识库中的 "part_6.txt" 为例，内容为：



Reader 使用方法



1. `from lazyllm.tools.rag import Document`
2. `doc = Document(dataset_path="your_doc_path")`
3. `data = doc._impl._reader.load_data(input_files=["your_doc_path/part_6.txt"])`
4. `print(f" DocNode: {data[6]}")`
5. `print(f"Dict: {data[6].to_dict()}")`

Reader是通过 `load_data` 函数来加载数据的，`reader`是 `Document`的一个内部对象，所以用下划线开头。

把文档中的内容解析出来，并封装成统一的格式供后面的检索模块使用。

DocNode: DocNode(id: cefd8331-dda6-49cd-a91d-de39ed69833f, group: lazyllm_root, content: MIT OpenCourseWare是由麻省理工学院在2002年创始的一项计划。它的目标是在2007年年底以前，让所有大学生和...) parent: , children: {}

Dict: {'content': ' MIT OpenCourseWare是由麻省理工学院在2002年创始的一项计划。它的目标是在2007年年底以前，让所有大学生和...', 'embedding': {}, 'metadata': {}}



Reader 使用方法



github.com/LazyAGI/Tutorial

LazyAGI / Tutorial

Type to search

Code Issues Pull requests Actions Projects Security Insights

Tutorial Public

Edit Pins Watch 1 Fork 6 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

wzh1994 Create README.md dde707a · 6 hours ago 19 Commits

rag	Create README.md	6 hours ago
.flake8	Code for tutorial-chapter: 3,9,10,18	last week
LICENSE	Initial commit	last week
README.md	Initial commit	last week

README Apache-2.0 license

Tutorial

Tutorial of LazyLLM

About

Tutorial of LazyLLM

Readme

Apache-2.0 license

Activity

Custom properties

0 stars

1 watching

6 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors 6

Languages

Python 100.0%

https://github.com/LazyAGI/Tutorial/graphs/contributors



LazyLLM 支持的 Reader



通过前面的介绍，我们了解了 Reader 模块的作用。既然 Reader 模块的作用是把知识库里面的内容解析出来，那么我们会想到一个问题。计算机中文档的存储格式是多种多样的，那 LazyLLM 支持哪些类型呢？



目前 LazyLLM 中的 Reader 默认支持的文档格式包括 pdf、doc、hwp、ppt、ipynb、epub、markdown、mbox、csv、excel、image、MP3、MP4。



为什么要自定义 Reader ?

那如果知识库中出现了 LazyLLM 不支持的文档格式就不能使用 LazyLLM 了吗?

——当然不是。

对于一些 LazyLLM 中 Reader 没有支持的文档格式, 或者 LazyLLM 中 Reader 输出格式不符合自己需求的话, 我们就可以根据实际需求来定义一个满足自己需求的 Reader, 然后让 LazyLLM 来使用即可。这里假设我们知识库中存在 `html` 格式的文档, 以一个网页为例进行说明, 网页数据为右图所示。



大模型入门到进阶：什么是 RAG？为什么需要 RAG？RAG 的流程

原创

AI老猴子

已于 2024-09-12 12:54:37 修改

阅读量5k

收藏 25

点赞数 22

版权

文章标签：

人工智能

大模型

AI大模型

ai

大模型入门

RAG

学习



开放原子开发者工作坊 文章已被社区收录

加入社区

一、什么是 RAG？

RAG 全称 Retrieval-Augmented Generation, 翻译成中文是检索增强生成。检索指的是检索外部 `知识库`, 增强生成指的是将检索到的知识送给 `大语言模型` 以此来优化大模型的生成结果, 使得大模型在生成更精确、更贴合上下文答案的同时, 也能有效减少产生误导性信息的可能。

二、为什么需要 RAG？

之所以需要 RAG, 是因为 `大语言模型` 本身存在一些局限性。

1. 时效性

模型的训练是基于截至某一时间点之前的数据集完成的。这意味着在该时间点之后发生的任何事件、新发现、新趋势或数据更新都不会反映在模型的知识库中。例如, 我的训练数据在 2023 年底截止, 之后发生的事情我都无法了解。另外, 大型模型的训练涉及巨大的计算资源和时间。这导致频繁更新模型以包括最新信息是不现实的, 尤其是在资源有限的情况下。

2. 覆盖性

虽然大模型的 `训练数据集` 非常庞大, 但仍可能无法涵盖所有领域的知识或特定领域的深度信息。例如, 某些专业的医学、法律或技术问题可能只在特定的文献中被详细讨论, 而这些文献可能未被包括在模型的训练数据中。另外, 对于一些私有数据集, 也是没有被包含在训练数据中的。当我们问的问题的答案没有包含在大模型的训练数据集中时, 这时候大模型在回答问题时便会出现幻觉, 答案也就缺乏可信度。



为什么要自定义 Reader ?



首先，我们需要把当前网页数据保存成 html 格式文件，我们可以手动进行数据保存，也可以通过代码进行保存。
下面代码为把网页数据保存成文件的过程。

```
1. import requests
   # 当前网页url
2. url = "https://blog.csdn.net/star_nwe/article/details/141174167"
3. headers = {
4.     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/131.0.0.0 Safari/537.36"
5. }
6. response = requests.get(url, headers=headers)
7. if response.status_code == 200:
8.     with open("webPage.html", "w", encoding='utf-8') as file:
9.         file.write(response.text)
10.    print("Webpage downloaded successfully!")
11. else:
12.    print(f"Failed to download webpage. Status code: {response.status_code}")
```

通过运行上面的代码可以在当前目录中生成一个 `webPage.html` 的文件。



为什么要自定义 Reader ?



我们先看一下如果使用 LazyLLM 中默认的 Reader 解析出来是什么内容，使用方式如下：

1. `from lazyllm.tools.rag import Document`
2. `doc = Document(dataset_path="your_doc_path")`
3. `data = doc._impl._reader.load_data(input_files=["webPage.html"])`
4. `print(f"text: {data[0].text}")`

从右侧的结果，可以发现和我们的预期差别很大，里面除了网页的内容之外，**还包括 html 里的各种标签**。而我们预期的是 Reader 解析出来的数据应该只是网页上文字，而不应该包含 html 的标签。

```
1 text:      <!DOCTYPE html>
2 <html lang="zh-CN">
3
4 <head>
5     <meta charset="utf-8">
6     <link rel="canonical" href="https://blog.csdn.net/star_nwe/article/de
7     <meta http-equiv="content-type" content="text/html; charset=utf-8">
8     <meta name="renderer" content="webkit"/>
9     <meta name="force-rendering" content="webkit"/>
10    <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1"/>
11    <meta name="viewport" content="width=device-width, initial-scale=1.0,
12    <meta name="report" content='{ "pid": "blog", "spm": "1001.2101"}'>
13    <meta name="referrer" content="always">
14    <meta http-equiv="Cache-Control" content="no-siteapp" /><link rel="al
15    <meta name="shenma-site-verification" content="5a59773ab8077d4a62bf46
16    <meta name="applicable-device" content="pc">
17    <link href="https://g.csdnimg.cn/static/logo/favicon32.ico" rel="sh
18    <title>大模型入门到进阶：什么是 RAG？为什么需要 RAG？RAG 的流程-CSDN博客</tit
```



目录



1. 上节回顾



2. Reader简介



3. 自定义Reader



4. 特殊情况下的Reader



5. 实战：复杂PDF解析



定义html读取函数



既然 LazyLLM 默认的 Reader 不能满足我们的需求，那我们就需要定义一个我们自己的 Reader。因为解析 html 格式的文档时，需要使用 `lxml` 工具包，所以需要先安装这个工具包，安装命令如下：

```
pip install lxml
```

安装完 lxml 之后，就可以定义 Reader 了。

```
1. from lazyllm.tools.rag import DocNode
2. from bs4 import BeautifulSoup

3. def processHtml(file, extra_info=None):
4.     text = ""
5.     with open(file, 'r', encoding='utf-8') as f:
6.         data = f.read()
7.         soup = BeautifulSoup(data, 'lxml')
8.         for element in soup.stripped_strings:
9.             text += element + '\n'

10.     node = DocNode(text=text, metadata=extra_info or {})
11.     return [node]
```

我们先使用 BeautifulSoup 包对网页内容解析，把解析出来的文本再通过 DocNode 类实例化成一个 node 对象，然后再以 list 的格式返回（即 LazyLLM 要求的 Reader 返回类型）



注册函数到reader中



1. `from lazyllm.tools.rag import Document`

2. `doc = Document(dataset_path="your_doc_path")`

3. `doc.add_reader("*.html", processHtml)`

4. `data = doc._impl._reader.load_data(input_files=["webPage.html"])`

5. `print(f"data: {data}")`

6. `print(f"text: {data[0].text}")`

“*.html” 表示对所有的
html格式的文档生效

```
1 text: 大模型入门到进阶: 什么是 RAG? 为什么需要 RAG? RAG 的流程-CSDN博客
2 大模型入门到进阶: 什么是 RAG? 为什么需要 RAG? RAG 的流程
3 置顶
4 大模型微调部署
5 已于 2024-09-12 12:54:37 修改
6 阅读量2.1w
7 收藏
8 74
9 点赞数
10
11 文章标签:
12 人工智能
13 大模型
14 AI大模型
15 ai
16 大模型入门
```

大模型入门到进阶: 什么是 RAG? 为什么需要 RAG? RAG 的流程

AI数据子 已于 2024-09-12 12:54:37 修改 阅读量5k 收藏 25 点赞数 22 版权

文章标签: 人工智能 大模型 AI大模型 大模型入门 RAG 学习

开放原子开发者工作坊 文章已被社区收录 加入社区

一、什么是 RAG?

RAG 全称 Retrieval-Augmented Generation, 翻译成中文是检索增强生成。检索指的是检索外部知识库, 增强生成指的是将检索到的知识送给大语言模型, 以此来优化大模型的生成结果, 使得大模型在生成更精确、更贴合上下文答案的同时, 也能有效减少产生误导性信息的可能。

二、为什么需要 RAG?

之所以需要 RAG, 是因为大语言模型本身存在一些局限性。

1.时效性

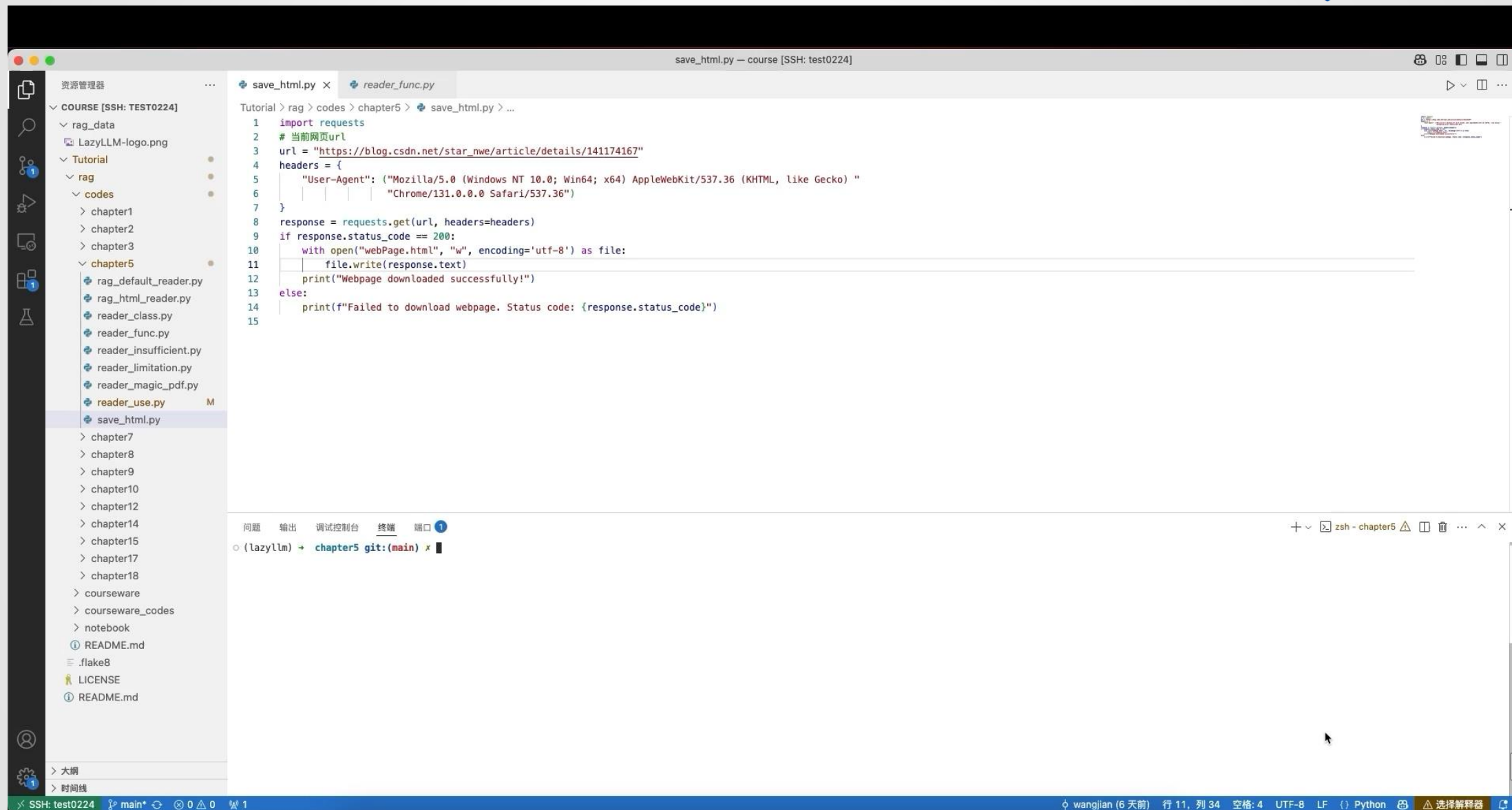
模型的训练是基于截至某一时间点之前的数据集完成的。这意味着在该时间点之后发生的任何事件、新发现、新趋势或数据更新都不会反映在模型的知识库中。例如, 我的训练数据在 2023 年底截止, 之后发生的事情我都无法了解。另外, 大型模型的训练涉及巨大的计算资源和时间, 这导致频繁更新模型以包括最新信息是不现实的, 尤其是在资源有限的情况下。

2.覆盖性

虽然大模型的训练数据集非常大, 但仍可能无法涵盖所有领域的知识或特定领域的深度信息。例如, 某些专业的医学、法律或技术问题可能只在特定的文献中被详细讨论, 而这些文献可能未被包括在模型的训练数据中。另外, 对于一些私有数据集, 也是没有被包含在训练数据中的。当我们问的问题的答案没有包含在大模型的训练数据集中时, 这时候大模型在回答问题时就会出现幻觉, 答案也就缺乏可信度。



效果展示



构建基于自定义HTML解析的 RAG 应用



```
1. import os
2. import lazyllm
3. from lazyllm.tools.rag import Document
4. from lazyllm import SentenceSplitter, pipeline, parallel, Retriever, Reranker, bind
5. prompt = ('You will play the role of an AI Q&A assistant and complete a dialogue task. '
            'In this task, you need to provide your answer based on the given context and question.' )

6. documents = Document(dataset_path=os.path.join(os.getcwd(), "rag_data" ),
                        embed=lazyllm.OnlineEmbeddingModule(source="glm", embed_model_name="embedding-2"), manager=False)
7. documents.create_node_group(name="sentences", transform=SentenceSplitter, chunk_size=1024, chunk_overlap=100)

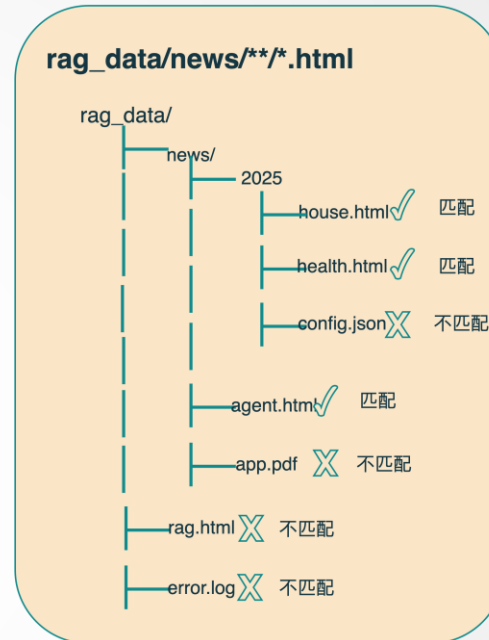
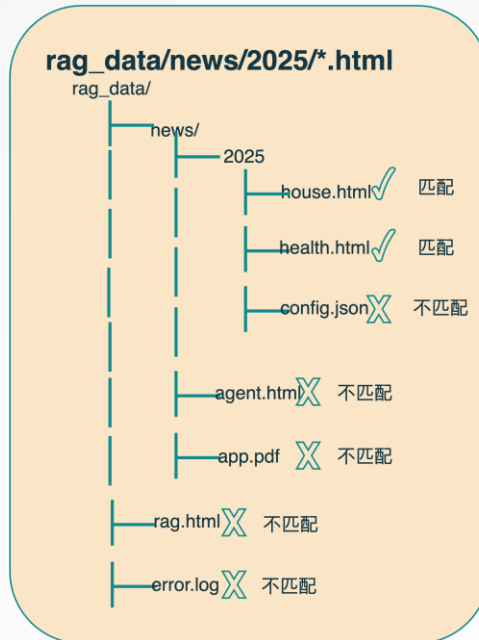
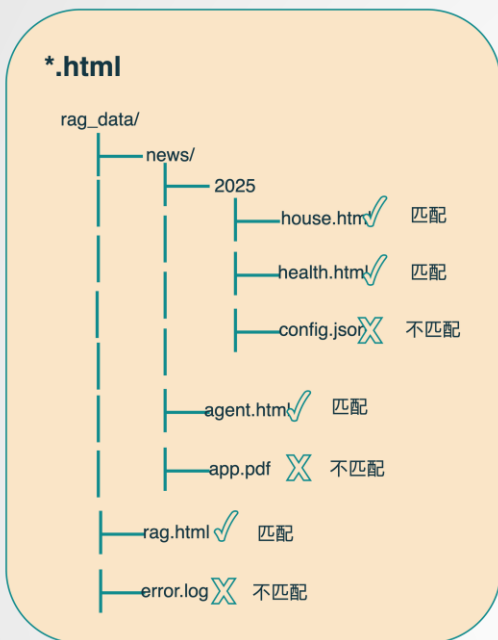
8. documents.add_reader("*.html", processHtml)

9. documents.create_node_group(name="sentences", transform=SentenceSplitter, chunk_size=1024, chunk_overlap=100)
10. with pipeline() as ppl:
11.     with parallel().sum as ppl.prl:
12.         prl.retriever1 = Retriever(documents, group_name="sentences", similarity="cosine", topk=3)
13.         prl.retriever2 = Retriever(documents, "CoarseChunk", "bm25_chinese", 0.003, topk=3)
14.     ppl.reranker = Reranker( "ModuleReranker" , model=lazyllm.OnlineEmbeddingModule(type= "rerank" , source= "glm" ,
        embed_model_name="rerank"), topk=1, output_format='content', join=True) | bind(query=ppl.input)
15.     ppl.formatter = (lambda nodes, query: dict(context_str=nodes, query=query)) | bind(query=ppl.input)
16.     ppl.llm = lazyllm.OnlineChatModule(source='glm', model="glm-4", stream=False).prompt(lazyllm.ChatPrompter(prompt,
        extra_keys=["context_str"]))
```



Reader 的pattern介绍

这里我们将更深入地讲解一下Reader 的注册方式。



● Reader 是通过 **key - value** 的方式进行注册：

- **key** 是用来说明 Reader 可以处理的数据格式，可以是直接指定对应的文件，也可以指定 Reader 要处理的数据模式，即可以使用通配符 * 来指定匹配规则，例如 “*.html” 表示可以处理任何以 “html” 为后缀的文件，“aaa/bbb/*.html” 表示要处理的是 “aaa/bbb” 目录下的任何以 “html” 为后缀的文件，“aaa/bbb/**/*.html” 表示要处理的是 “aaa/bbb” 目录下所有目录中任何以 “html” 为后缀的文件。
- **value** 就是对应的 Reader。



自定义HTML解析的RAG 应用-展示



```
rag_html_reader.py 5 x rag_default_reader.py 4
Tutorial1 > rag > codes > chapter5 > rag_html_reader.py > ...
1 import os
2 import lazyllm
3 from lazyllm.tools.rag import Document
4 from lazyllm import SentenceSplitter
5 from lazyllm.tools.rag import DocNode
6 from bs4 import BeautifulSoup
7 from lazyllm import pipeline, parallel, Retriever, Reranker, bind
8
9
10 def processHtml(file, extra_info=None):
11     text = ''
12     with open(file, 'r', encoding='utf-8') as f:
13         data = f.read()
14         soup = BeautifulSoup(data, 'xml')
15         for element in soup.stripped_strings:
16             text += element + '\n'
17     node = DocNode(text=text, metadata=extra_info or {})
18     return [node]
19
20
21 documents = Document(dataset_path=os.path.join(os.getcwd(), "rag_data"),
22                      embed=lazyllm.OnlineEmbeddingModule(source="qwen"), manager=False)
23 documents.add_reader("*.html", processHtml)
24 documents.create_node_group(name="sentences", transform=SentenceSplitter, chunk_size=1024, chunk_overlap=100)
25 prompt = ('You will play the role of an AI Q&A assistant and complete a dialogue task. '
26          'In this task, you need to provide your answer based on the given context and question.')
27 with pipeline() as ppl:
28     with parallel().sum as ppl.prl:
29         ppl.prl.retriever1 = Retriever(documents, group_name="sentences", similarity="cosine", topk=3)
30         ppl.prl.retriever2 = Retriever(documents, "CoarseChunk", "bm25_chinese", 0.003, topk=3)
31         ppl.reranker = Reranker("ModuleReranker",
32                                model=lazyllm.OnlineEmbeddingModule(type="rerank", source="qwen"),
33                                topk=1, output_format='content', join=True) | bind(query=ppl.input)
34         ppl.formatter = (lambda nodes, query: dict(context_str=nodes, query=query)) | bind(query=ppl.input)
35         ppl.llm = lazyllm.OnlineChatModule(source='sensenova',
36                                           model="SenseChat-5",
37                                           stream=False).prompt(lazyllm.ChatPrompter(prompt, extra_keys=["context_str"]))
38
39 print(ppl("全国住房城乡建设工作会议的主要内容"))
40
```

问题 9 输出 调试控制台 终端 端口 18

o (lazyllm) + chapter5 git:(courseware_codes) x

zsh courseware_c
zsh chapter5

SSH: test0224 courseware_codes* 0 9 18 尚未提交 行 22, 列 88 空格: 4 UTF-8 LF Python 3.10.9 (lazyllm: conda)

目录



1. 上节回顾



2. Reader简介



3. 自定义Reader



4. 特殊情况下的Reader



5. 实战：复杂PDF解析



多级 Reader 注册

支持向文档模块中注册自定义的Read !

```
docs1 = Document(...)
docs2 = Document(...)
```

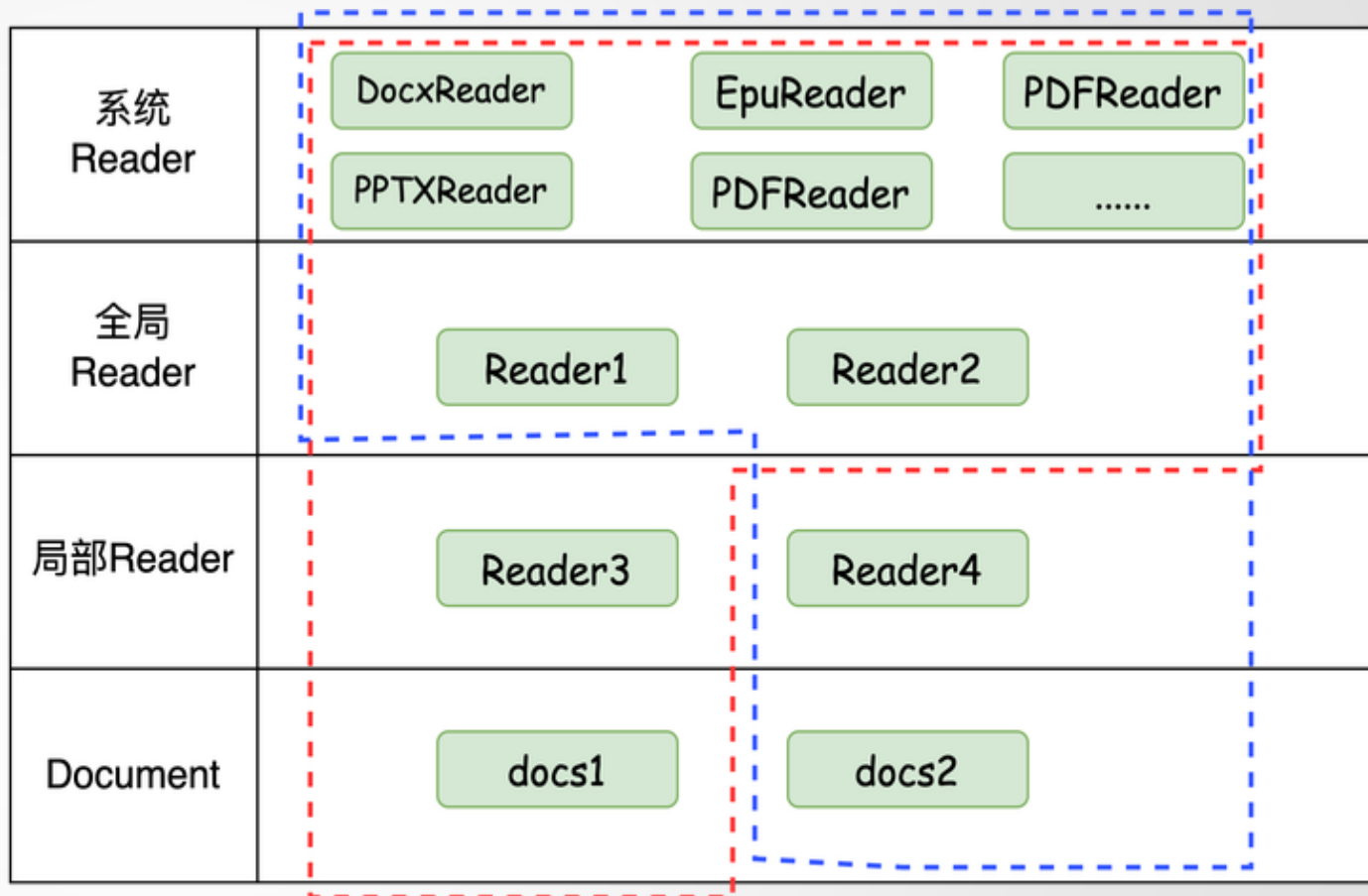
注册全局Reader

```
Document.add_reader(pattern, Reader1)
Document.add_reader(pattern, Reader2)
```

注册局部Reader

```
docs1.add_reader(pattern, Reader1)
docs1.add_reader(pattern, Reader2)
```

**/docs1/*.pdf
**/law*.pdf
*.pdf

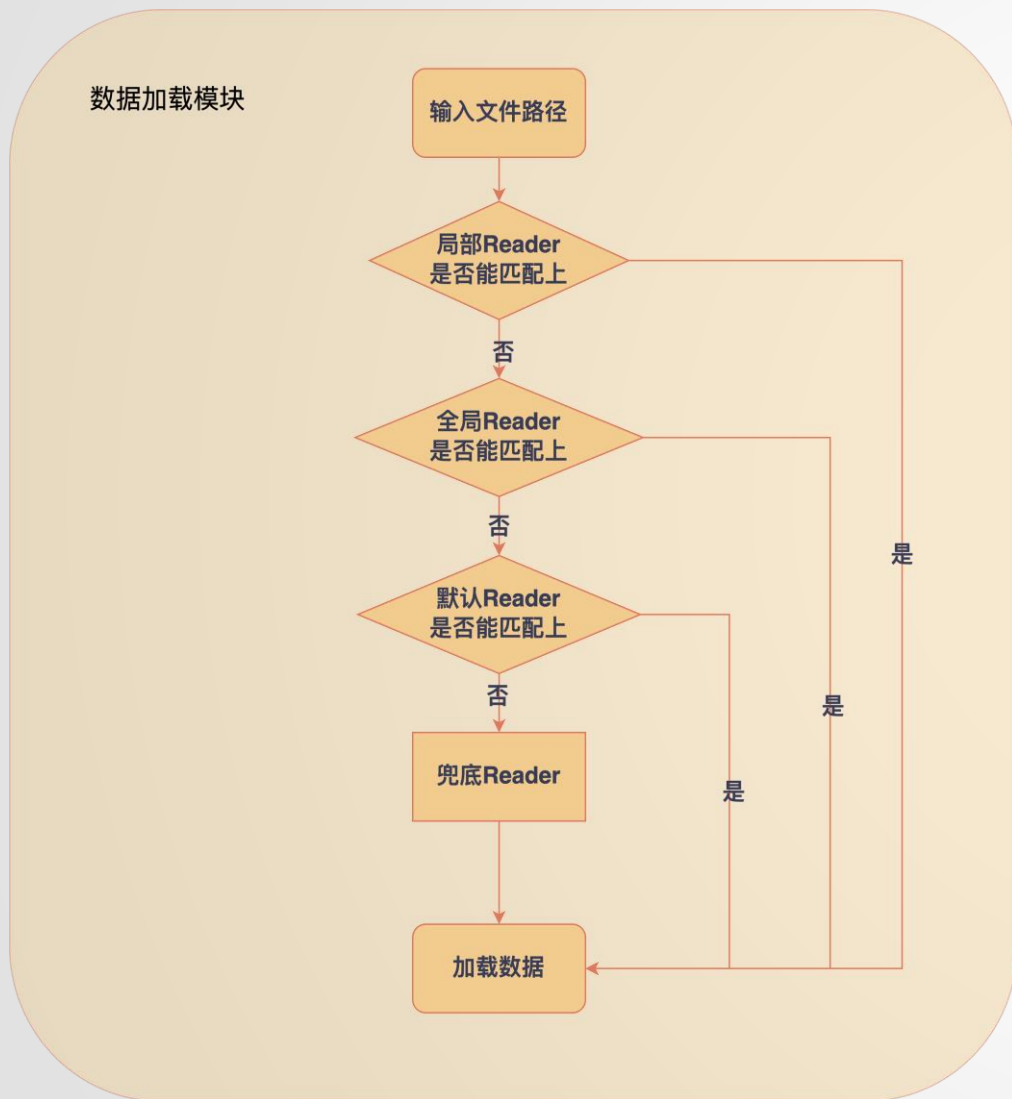


优先级：局部Reader > 全局Reader > 系统Reader

“通过对象注册的 Reader” > “通过类注册的Reader” > “LazyLLM 默认提供的 Reader”



多级 Reader 注册



系统 Reader	DocxReader PPTXReader	EpuReader PDFReader	PDFReader
全局 Reader	Reader1	Reader2	
局部Reader	Reader3	Reader4	
Document	docs1	docs2	

“通过对象注册的 Reader” > “通过类注册的Reader” > “LazyLLM 默认提供的 Reader”

多级 Reader 注册 效果展示



```
1. from lazyllm import Document
2.
3. def reader1(path, extra_info=None): print(f'reader1 read {path}'); return []
4. def reader2(path, extra_info=None): print(f'reader2 read {path}'); return []
5. def reader3(path, extra_info=None): print(f'reader3 read {path}'); return []
6.
7. pdf_file, txt_file, html_file = '/path/to/a.pdf' , '/path/to/a.txt' , '/path/to/a.html'
8. docs1, docs2, docs3 = Document('/path'), Document('/path'), Document('/path')
9. docs1.add_reader('*.pdf', reader1)
10. docs2.add_reader('*.pdf', reader2)
11. Document.add_reader('*.pdf', reader3)
12. Document.add_reader('*.txt', reader3)
13.
14. # 匹配成功,读局部配置
15. docs1._impl._reader.load_data(input_files=[pdf_file])
16. >> reader1 read /path/to/a.pdf
17. docs2._impl._reader.load_data(input_files=[pdf_file])
18. >> reader2 read /path/to/a.pdf
19.
20. ## 局部匹配失败,读全局配置
21. docs3._impl._reader.load_data(input_files=[pdf_file])
22. >> reader3 read /path/to/a.pdf
23. docs1._impl._reader.load_data(input_files=[txt_file])
24. >> reader3 read /path/to/a.txt
25. docs2._impl._reader.load_data(input_files=[txt_file])
26. >> reader3 read /path/to/a.txt
27.
28. # 全局匹配失败,走默认配置
29. docs1._impl._reader.load_data(input_files=[html_file])
30. >> [<Node id=d753dc21-1d70-412f-a91d-f6c123321c77>]
```

Folder

```
|—— a.pdf
|—— a.txt
|—— a.html
```



多级 Reader 注册 – 原理



学到了！深入体验LazyLLM，学习更多python编程的“奇技淫巧”

设定全局Reader

```
Document.add_reader(..., Reader1)
Document.add_reader(..., Reader2)
```

设定局部Reader

```
doc1.add_reader(..., Reader3)
doc2.add_reader(..., Reader4)
```

```
1 class DynamicDescriptor:
2     class Impl:
3         def __init__(self, func, instance, owner):
4             self._func, self._instance, self._owner = func, instance, owner
5
6         def __call__(self, *args, **kw):
7             return self._func(self._instance, *args, **kw) if self._instance else self._func(self._owner, *args, **kw)
8
9         def __repr__(self): return repr(self._func)
10        __doc__ = property(lambda self: self._func.__doc__)
11
12        @_doc_.setter
13        def __doc__(self, value): self._func.__doc__ = value
14
15    def __init__(self, func):
16        self._func__ = func
17
18    def __get__(self, instance, owner):
19        return DynamicDescriptor.Impl(self._func__, instance, owner)
```

```
1 class Document(object):
2
3     @DynamicDescriptor
4     def add_reader(...)
```



基于类来自定义 Reader



但是当 Reader 需要在多个调用之间**保持状态**（如缓存、计数器、连接池等）或者需要在初始化时进行**复杂的对象创建**（如数据库连接、模型加载等）以及解析过程复杂，需要拆分多个方法的。这时再使用函数的形式就不合适了。我们就得使用基于类的方式来定义 Reader 了。并且因为 Reader 有不同类型的子类，所以基于类的形式可以扩展不同类型的解析器。

- **定义个类**：这里我们以一个给图片提取摘要的例子来说明，因为这个 Reader 需要使用到 `torch`、`transformers`、`sentencepiece` 和 `Pillow` 库，如果环境中没有需要提前安装，安装命令如下：

```
pip install torch transformers sentencepiece Pillow
```

然后我们开始定义我们需要的 Reader，因为这个 Reader 需要导入模型，所以要把模型加载放到初始化函数中，这样在多次使用时，就不用每次都加载一遍了，这种情况明显定义函数是不合适的。



基于类来自定义 Reader



```
1. from lazylm.tools.rag.readers import ReaderBase
2. from lazylm.tools.rag.readers.readerBase import infer_torch_device
3. ... ..
4. class ImageDescriptionReader(ReaderBase):
5.     def __init__(self, parser_config: Optional[Dict] = None, prompt: Optional[str] = None) -> None:
6.         super().__init__()
7.         if parser_config is None:
8.             ... ..
9.             processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-large")
10.             model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-large")
11.             self.parser_config = {"processor": processor, "model": model, "device": device, "dtype": dtype}
12.         ... ..
13. ... ..
```

⚠ 类要继承自ReaderBase

⚠ 重写 load_data函数

```
14. def load_data(self, file: Path, extra_info: Optional[Dict] = None) -> List[DocNode]:
15.     image = Image.open(file)
16.     if image.mode != "RGB": image = image.convert("RGB")
17.     model = self._parser_config['model']
18.     ... ..
19.     model.to(device)
20.     inputs = processor(image, self._prompt, return_tensors="pt").to(device, dtype)
21.     out = model.generate(**inputs)
22.     text_str = processor.decode(out[0], skip_special_tokens=True)
23.     return [DocNode(text=text_str, metadata=extra_info or {})]
```

⚠ 返回list[DocNode]



基于类来自定义 Reader

基于类的 Reader 注册方法和**基于函数的完全相同**！

1. `from lazyllm.tools.rag import Document`
2. `doc = Document(dataset_path="your_doc_path")`
3. `doc.add_reader("*.png", ImageDescriptionReader)`
4. `data = doc._impl._reader.load_data(input_files=["cmrc2018_path/LazyLLM-logo.png"])`
5. `print(f"data: {data}")`
6. `print(f"text: {data[0].text}")`



Node 中的文本内容，生成的图片描述文本内容，也符合预期。

text: a close up of a cat sitting on a chair next to a computer

基于类来自定义 Reader



reader_class.py — course [SSH: test0224]

资源管理器

- COURSE [SSH: TEST0224]
 - rag_data
 - LazyLLM-logo.png
 - webPage.html
 - Tutorial
 - rag
 - codes
 - chapter1
 - chapter2
 - chapter3
 - chapter5
 - rag_default_reader.py
 - rag_html_reader.py
 - reader_class.py M
 - reader_func.py M
 - reader_insufficient.py
 - reader_limitation.py
 - reader_magic_pdf.py
 - reader_use.py M
 - save_html.py
 - chapter7
 - chapter8
 - chapter9
 - chapter10
 - chapter12
 - chapter14
 - chapter15
 - chapter17
 - chapter18
 - courseware
 - courseware_codes
 - notebook
 - README.md
 - .flake8
 - LICENSE
 - README.md
 - 大纲
 - 时间线

save_html.py reader_func.py M reader_class.py M X

Tutorial > rag > codes > chapter5 > reader_class.py > ...

```
12 class ImageDescriptionReader(ReaderBase):
13     self._prompt = prompt
14
15
16
17
18
19
20
21
22
23
24
25
26 def _load_data(self, file: Path, extra_info: Optional[Dict] = None) -> List[DocNode]:
27     image = Image.open(file)
28     if image.mode != "RGB":
29         image = image.convert("RGB")
30
31     model = self._parser_config['model']
32     processor = self._parser_config["processor"]
33
34     device = self._parser_config["device"]
35     dtype = self._parser_config["dtype"]
36     model.to(device)
37
38     inputs = processor(image, self._prompt, return_tensors="pt").to(device, dtype)
39
40     out = model.generate(**inputs)
41     text_str = processor.decode(out[0], skip_special_tokens=True)
42     return [DocNode(text=text_str, metadata=extra_info or {})]
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

doc = Document(dataset_path="/home/mnt/zhaoshe/course/rag_data")
doc.add_reader("*.png", ImageDescriptionReader)
data = doc._impl._reader.load_data(input_files=["/home/mnt/zhaoshe/course/rag_data/LazyLLM-logo.png"])
print(f"data: {data}")
print(f"text: {data[0].text}")

问题 输出 调试控制台 终端 端口 1

o {lazyllm} → chapter5 git:(main) x

zsh - chapter5

wangjian (6 天前) 行 44, 列 1 空格: 4 UTF-8 LF Python 选择解释器

目录



1. 上节回顾



2. Reader简介



3. 自定义Reader



4. 特殊情况下的Reader



5. 实战：复杂PDF解析



当前 Reader 的不足



平安证券

珀莱雅(603605.SH)

高质量稳健增长，双十一稳定发挥

推荐（维持）

股价：96.75 元

主要数据

行业	社会服务
公司网址	www.prlay.com
大股东/持股	侯军呈/34.46%
实际控制人	侯军呈,方爱琴
总股本(百万股)	396
流通A股(百万股)	396
流通B/H股(百万股)	
总市值(亿元)	383
流通A股市值(亿元)	382
每股净资产(元)	12.09
资产负债率(%)	34.2

行情走势图



相关研究报告

【平安证券】珀莱雅（603605.SH）*季报点评*H1经营稳健，管理层换届*推荐20240913

【平安证券】珀莱雅（603605.SH）*首次覆盖报告*大单品策略，驱动增长+敏捷执行，持续突破**推荐20240518

证券分析师

胡晓方 投资咨询资格证书
S1060524010002
HUOXIONGFANG722@pingan.com.cn



事项：

公司 2024 前三季度实现营收 69.66 亿元(+32.72%)，33.95%至 9.99 亿元。扣除非经常性损益后为 9.71 亿元(=2.53 元)。毛利率为 70.07%(-1.11pct)，净利率为 14.68 同比（下同）+42.29%至 32.32 亿元，销售费用率 46.3 费用-4.88%至 2.73 亿元，管理费用率 3.91%，-1.55 pct 研发费用率 2.04%，-0.41 pct；财务费用率-0.57%，+0 单季度营收同比增长 21.15%至 19.65 亿元，归母净利润 2.98 亿元。

平安观点：

■ 在美妆整体经营弱化的环境下，公司 Q3 获得高质量预期。根据国家统计局，社零中化妆品类 1-9 月 1 亿元，而公司三季度获得双 20%+的收入和归母净利润。同时三季度公司毛销差相较于上半年有所优化 69.82%，Q3 单季度为 70.71%（同比-1.95pct），46.78%，Q3 单季度为 45.40%（较上年同期+2.78 15.19%（同比-0.02pct）。

■ 步入 Q4 后，今年各电商平台双十一战线拉长，美公司旗下品牌表现亮眼。根据天猫大美妆官方公布月 21 日 20:00-10 月 23 日 24:00）快消榜珀莱雅第三分别为巴黎欧莱雅和兰蔻；美妆榜珀莱雅同样分别为兰蔻和巴黎欧莱雅。此前的天猫美妆预售阶段月 17 日 24:00），巴黎欧莱雅和修丽可位列第二。

■ 投资建议：美妆行业增速放缓，同时竞争激烈，公司以优质产品和运营抢占市场份额，获得稳健地高质量的双十一销售中稳定发挥。我们维持对公司 2024-2 略调整 2026 年预测，2024-2026 年对应归母净利润 23.08 亿元（2026 年原预测为 23.09 亿元），对应市值）分别约 24.7、20.1 和 16.6 倍 PE。公司作

	2022A	2023A	2024E
营业收入(百万元)	6,385	8,905	11,492
YOY(%)	37.8	39.5	29.1
净利润(百万元)	817	1,194	1,552
YOY(%)	41.9	46.1	30.0
毛利率(%)	69.7	69.9	70.3
净利率(%)	12.8	13.4	13.5
ROE(%)	23.2	27.4	28.9
EPS(摊薄/元)	2.06	3.01	3.92
P/E(倍)	46.9	32.1	24.7
P/B(倍)	11.0	8.9	7.2

平安证券

珀莱雅·公司季报点评

资产负债表	单位：百万元			
会计年度	2023A	2024E	2025E	2026E
流动资产	5,545	6,672	8,241	10,105
现金	4,011	4,777	5,959	7,397
应收票据及应收账款	352	409	495	590
其他应收款	82	134	162	193
预付账款	203	252	305	363
存货	797	983	1,179	1,393
其他流动资产	100	117	142	169
非流动资产	1,778	1,678	1,583	1,516
长期投资	114	84	59	39
固定资产	841	811	821	825
无形资产	405	371	337	304
其他非流动资产	419	412	366	349
资产总计	7,323	8,350	9,824	11,621
流动负债	2,120	2,308	2,710	3,155
短期借款	200	150	120	90
应付票据及应付账款	1,055	1,124	1,348	1,593
其他流动负债	865	1,034	1,242	1,472
非流动负债	803	616	429	255
长期借款	763	576	389	216
其他非流动负债	39	39	39	39
负债合计	2,923	2,924	3,139	3,410
少数股东权益	51	57	65	75
股本	397	396	396	396
资本公积	864	865	865	865
留存收益	3,089	4,108	5,359	6,875
归属母公司股东权益	4,350	5,369	6,620	8,136
负债和股东权益	7,323	8,350	9,824	11,621

现金流量表	单位：百万元			
会计年度	2023A	2024E	2025E	2026E
经营活动现金流	1,464	1,583	2,081	2,425
净利润	1,231	1,558	1,911	2,318
折旧摊销	92	160	170	147
财务费用	-59	-53	-85	-123
投资损失	17	30	25	20
营运资金变动	-10	-124	46	49
其他经营现金流	194	12	14	14
投资活动现金流	-476	-102	-114	-114
资本支出	225	90	100	100
长期投资	-13	0	0	0
其他投资现金流	-688	-192	-214	-214
筹资活动现金流	-460	-716	-785	-873
短期借款	-0	-50	-30	-30
长期借款	35	-187	-187	-174
其他筹资现金流	-495	-479	-568	-669
现金净增加额	529	766	1,182	1,438

资料来源：同花顺 iFinD，平安证券研究所

利润表	单位：百万元			
会计年度	2023A	2024E	2025E	2026E
营业收入	8,905	11,492	13,897	16,560
营业成本	2,677	3,418	4,097	4,842
税金及附加	91	106	129	153
营业费用	3,972	5,298	6,421	7,667
管理费用	455	550	662	769
研发费用	174	230	278	331
财务费用	-59	-53	-85	-123
资产减值损失	-108	-69	-97	-116
信用减值损失	-10	-11	-14	-17
其他收益	45	90	100	90
公允价值变动收益	0	0	0	0
投资净收益	-17	-30	-25	-20
资产处置收益	-1	0	0	0
营业利润	1,503	1,922	2,359	2,858
营业外收入	4	5	0	4
营业外支出	12	3	0	0
利润总额	1,495	1,924	2,359	2,862
所得税	265	366	448	544
净利润	1,231	1,558	1,911	2,318
少数股东损益	37	6	8	10
归属母公司净利润	1,194	1,552	1,903	2,308
EBITDA	1,528	2,031	2,444	2,886
EPS（元）	3.01	3.92	4.80	5.83

主要财务比率	2023A	2024E	2025E	2026E
成长能力				
营业收入(%)	39.5	29.1	20.9	19.2
营业利润(%)	42.1	27.9	22.8	21.1
归属于母公司净利润(%)	46.1	30.0	22.6	21.3
获利能力				
毛利率(%)	69.9	70.3	70.5	70.8
净利率(%)	13.4	13.5	13.7	13.9
ROE(%)	27.4	28.9	28.7	28.4
ROIC(%)	66.2	70.9	76.2	88.0

偿债能力	2023A	2024E	2025E	2026E
资产负债率(%)	39.9	35.0	32.0	29.3
净负债率(%)	-69.3	-74.6	-81.5	-86.4
流动比率	2.6	2.9	3.0	3.2
速动比率	2.1	2.3	2.4	2.6
营运能力				
总资产周转率	1.2	1.4	1.4	1.4
应收账款周转率	25.3	28.1	28.1	28.1
应付账款周转率	2.6	3.4	3.4	3.4
每股指标(元)				
每股收益(最新摊薄)	3.01	3.92	4.80	5.83
每股经营现金流(最新摊薄)	3.70	4.00	5.25	6.12
每股净资产(最新摊薄)	10.85	13.42	16.58	20.40
估值比率				
P/E	32.1	24.7	20.1	16.6
P/B	8.9	7.2	5.8	4.7
EV/EBITDA	24.4	17.4	14.0	11.4

请通过合法途径获取本公司研究报告，如经未经许可的渠道获得研究报告，请慎重使用并注意阅读研究报告尾页的声明内容。

3 / 4

ROE(%) 23.2 27.4 28.9 28.7 28.4
EPS(摊薄/元) 2.06 3.01 3.92 4.80 5.83
P/E(倍) 46.9 32.1 24.7 20.1 16.6
P/B(倍) 11.0 8.9 7.2 5.8 4.7

证券研究报告

text: 珀莱雅·公司季报点评



引入高性能开源工具



针对上面的问题，我们可以找一些效果更好符合需求的开源工具接入进来。我们这里以 **MinerU** 工具为例进行介绍。我们已经为 **MinerU** 提供了专门的接入组件，无需额外定制，即可顺畅集成。目前提供一键启动的 **MinerU** 服务端（server）以及配套的 PDF 客户端。

使用流程如下：先在本地启动 **MinerU** 解析服务，再通过接入 `MineruPDFReader` 获取解析后的文档内容。

1. 启动 MinerU 服务

```
lazyllm deploy mineru  
[--port <port>]  
[--cache_dir <cache_dir>]  
[--image_save_dir <image_save_dir>]  
[--model_source <model_source>]
```

参数说明

- **--port**: 服务端口号，默认随机分配。
- **--cache_dir**: 文档解析缓存目录，设置后相同文档无需重复解析，默认 **None**。
- **--image_save_dir**: 图片输出目录，设置后可保存文档内提取的图片，默认 **None**。
- **--model_source**: 模型来源，可选 **huggingface** 或 **modelscope**，默认 **huggingface**。

!注意：在启动服务之前，请先安装 MinerU 依赖：`lazyllm install mineru`

为确保解析结果稳定，当前固定 MinerU 版本为 2.5.4。服务运行所需资源请参考 [MinerU](<https://github.com/opendatalab/MinerU>) 官方文档。



验证开源工具的效果



2. 接入MineruPDFReader

在成功启动MinerU服务后，可通过 MineruPDFReader 组件接入实现利用MinerU进行PDF文档解析。

1. `from lazyllm.tools.rag import Document`
2. `from lazyllm.tools.rag.readers import import MineruPDFReader`
3. `doc = Document(dataset_path= "your_doc_path" )`

注册 PDF 解析器，url 替换为已启动的 MinerU 服务地址

4. `documents.add_reader( "*.pdf" , MineruPDFReader(url="http://127.0.0.1:8888"))`
5. `data = doc._impl._reader.load_data(input_files=[ "平安证券-珀莱雅.pdf" ])`



验证开源工具的效果



• 默认解析器解析效果:

投资建议:美妆行业增速放缓,同时竞争激烈,公司继续坚持大单品策略,以优质产品和运营抢占市场份额,获得稳健地高质量增长,且在截止目前的双十一销售中稳定发挥。我们维持对公司 2024-2025 年的业绩预测,并略调整 2026 年预测,2024-2026 年对应归母净利润分别为 15.52、19.03、23.08 亿元(2026 年原预测为 23.09 亿元),对应当前市值(2024-10-28 市值)分别约 24.7、20.1 和 16.6 倍 PE。公司作为国货龙头,近年来形

2022A 2023A 2024E 2025E 2026E
营业收入(百万元) 6,385 8,905 11,492 13,897 16,560

YOY(%) 37.8 39.5 29.1 20.9 19.2

净利润(百万元) 817 1,194 1,552 1,903 2,308

YOY(%) 41.9 46.1 30.0 22.6 21.3

毛利率(%) 69.7 69.9 70.3 70.5 70.8

净利率(%) 12.8 13.4 13.5 13.7 13.9

ROE(%) 23.2 27.4 28.9 28.7 28.4

EPS(摊薄/元) 2.06 3.01 3.92 4.80 5.83

P/E(倍) 46.9 32.1 24.7 20.1 16.6

P/B(倍) 11.0 8.9 7.2 5.8 4.7 请通过合法途径获取本公司研究报告,如经由未经许可的渠道获得研究报告,请慎重

- 自动还原文档的跨页结构
- 自动删除页眉、页脚、脚注、页码等元素

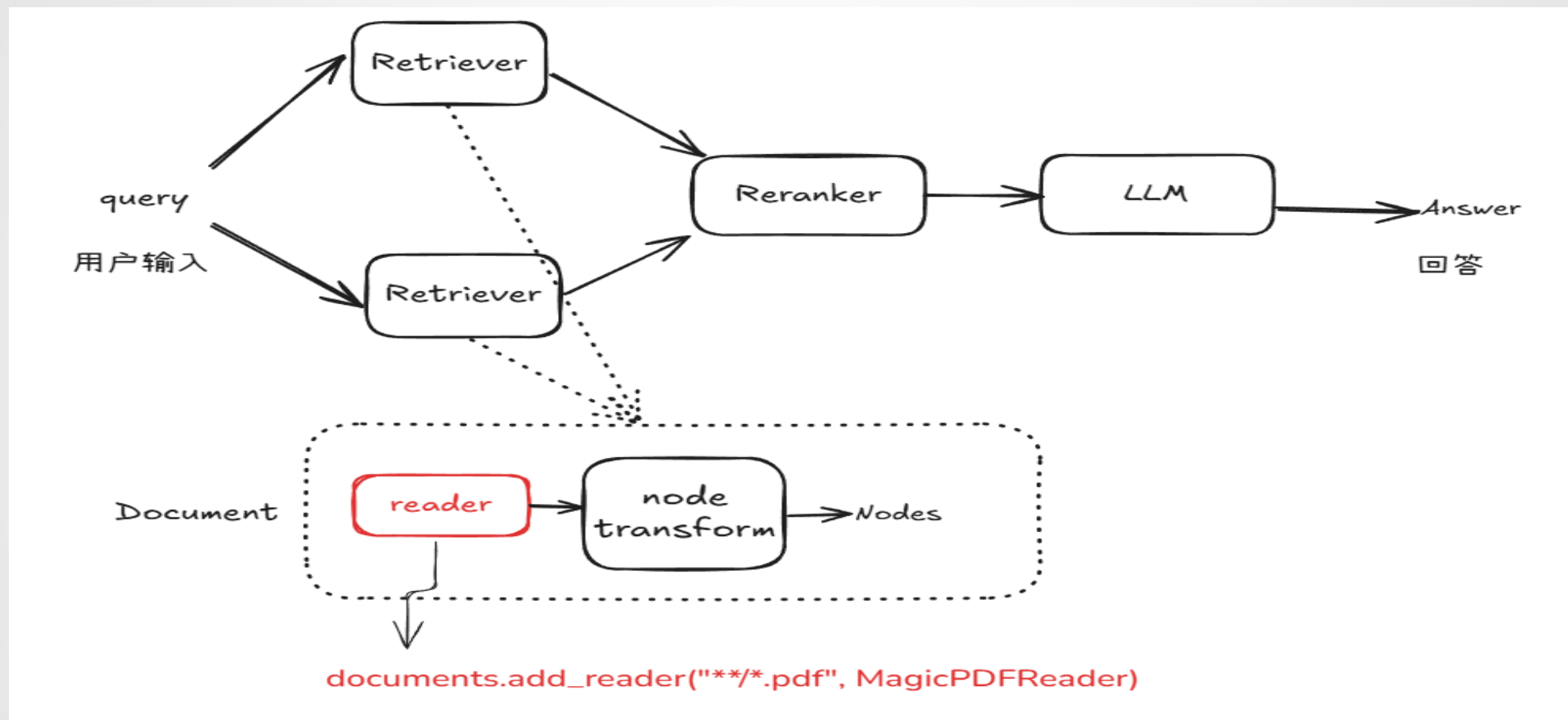
• MinerU解析效果:

投资建议:美妆行业增速放缓,同时竞争激烈,公司继续坚持大单品策略,以优质产品和运营抢占市场份额,获得稳健地高质量增长,且在截止目前的双十一销售中稳定发挥。我们维持对公司 2024-2025 年的业绩预测,并略调整 2026 年预测,2024-2026 年对应归母净利润分别为 15.52、19.03、23.08 亿元(2026 年原预测为 23.09 亿元),对应当前市值(2024-10-28市值)分别约 24.7、20.1 和 16.6 倍 PE。公司作为国货龙头,近年来形成了系统化、规范化、标准化的运营体系,日益参与到国际美妆集团的竞争中,未来有望进一步获取更大的大众美妆市场份额。目前公司估值水平处在历史中低位区间,结合当前估值水平及市场环境情况,维持“推荐”评级

	2022A	2023A	2024E	2025E	2026E
营业收入(百万元)	6,385	8,905	11,492	13,897	16,560
YOY (%)	37.8	39.5	29.1	20.9	19.2
净利润(百万元)	817	1,194	1,552	1,903	2,308
YOY(%)	41.9	46.1	30.0	22.6	21.3
毛利率(%)	69.7	69.9	70.3	70.5	70.8
净利率(%)	12.8	13.4	13.5	13.7	13.9
ROE(%)	23.2	27.4	28.9	28.7	28.4
EPS(摊薄/元)	2.06	3.01	3.92	4.80	5.83
P/E(倍)	46.9	32.1	24.7	20.1	16.6
P/B(倍)	11.0	8.9	7.2	5.8	4.7



在RAG中使用MinerU



```
doc = Document(dataset_path="your_doc_path")
doc.add_reader("*.pdf", MineruPDFReader)
retriever = Retriever(doc, ...)
```



在RAG中使用MinerU 效果展示



rag_mineru.py — course [SSH: test0224]

资源管理器

COURSE [SSH: TEST0224]

LazyRAG-MinerU

rag_pdf_data

src

.env

.gitignore

README.md

requirements.txt

rag_data

Tutorial

rag

codes

courseware

courseware_codes

lazyllm_mineru

parse_images

rag_pdf_data

603605.pdf

src

parser

rag

__pycache__

__init__.py

rag_mineru_output.py

rag_mineru.py

rag.py

utils.py

reader

README.md

notebook

README.md

.flake8

LICENSE

README.md

test.py

大纲

时间线

rag_mineru.py 5

```
Tutorial > rag > courseware_codes > lazyllm_mineru > src > rag > rag_mineru.py > ...
1  # -*- coding: utf-8 -*-
2  # flake8: noqa: F821
3
4  import lazyllm
5  from lazyllm import LOG
6  from lazyllm import pipeline, parallel, bind, OnlineEmbeddingModule, SentenceSplitter, Document, Retriever, Reranker
7  from src.reader.magic_pdf_reader import MagicPDFReader
8  from src.parser.magic_pdf_transform import MagicPDFTransform
9  from dotenv import load_dotenv
10 import os
11
12 load_dotenv()
13
14
15 prompt = 'You will play the role of an AI question-answering assistant and complete a conversation task in which you need to provide your answer based on the given context and question. Please
16
17 documents = Document(dataset_path=os.path.join(os.getcwd(), 'rag_pdf_data'), embed=OnlineEmbeddingModule(source="qwen"), manager=False)
18
19 documents.add_reader("**/*.pdf", MagicPDFReader)
20 documents.create_node_group(name="magic-pdf", transform=MagicPDFTransform)
21
22 with pipeline() as ppl:
23     with parallel().sum as ppl.prl:
24         prl.retriever1 = Retriever(documents, group_name="magic-pdf", similarity="cosine", topk=3)
25         prl.retriever2 = Retriever(documents, group_name="magic-pdf", similarity="bm25-chinese", topk=3)
26     ppl.reranker = Reranker("ModuleReranker",
27                             model=OnlineEmbeddingModule(type="rerank"), # model=OnlineEmbeddingModule(type="rerank")
28                             topk=1, output_format='content',
29                             io=True).bind(quer=prl.inout)
```

问题 5

输出

调试控制台

终端

端口 18

o (lazyllm) → lazyllm_mineru git:(add-mineru) x python ./src/rag/rag_mineru.py

行 1, 列 1 空格: 4 UTF-8 LF () Python 3.10.9 ('lazyllm': conda)





感谢聆听
Thanks for Listening

